

Lark: A Reinforcement Learning-based Multi-signal Fusion Adaptive Network Congestion Control Algorithm

Hang Tiancheng

Nanjing University of Posts and Telecommunications
1224045520@njupt.edu.cn

Abstract

Modern data center networks pose unprecedented challenges to TCP congestion control. Traditional loss-based algorithms suffer from delayed congestion signals and buffer bloat, while delay-based algorithms face unstable RTT measurements and poor fairness. This paper presents Lark—a reinforcement learning-based multi-signal fusion adaptive congestion control algorithm. The key contributions include: (1) a comprehensive 15-dimensional observation space that, for the first time, incorporates ECN state, congestion control state machine state, and congestion event types into the RL state representation; (2) a two-level priority congestion detection mechanism with a consecutive decrease protection that bounds multiplicative reductions to at most $D_{\max} = 3$; (3) a differentiated congestion response with type-specific window retention factors (loss: 0.70, ECN: 0.85, timeout: 0.50) and BDP-anchored slow-start threshold recovery; (4) RL-driven adaptive parameter tuning with three-level RTT gradient, reward exponential moving average, and sustained growth acceleration; (5) a hybrid rate-window fusion control strategy combining windowed max-filter BDP estimation with pipeline utilization-aware acceleration. Extensive experiments across 9 representative data center network scenarios on the ns-3 platform demonstrate that Lark achieves throughput comparable to NewReno/CUBIC (gap <1%) while reducing latency by 70% to 81% (from 1.9–2.8 ms to 0.44–0.84 ms). In-depth analysis of convergence, queue behavior, ECN utilization, fairness, and mixed traffic further validates Lark’s effectiveness.

Keywords: Congestion Control; Reinforcement Learning; Multi-signal Fusion; ECN; Data Center Network

1 Introduction

Modern data center network bandwidths have reached 100 Gbps with latencies at the microsecond scale, posing severe challenges to congestion control mechanisms [1]. Data center traffic exhibits a pronounced “Mice-Elephant” distribution: over 80% of flows are under 10 KB, but over 80% of bandwidth is consumed by less than 1% of large flows. Incast communication patterns prevalent in distributed systems can cause in-

stantaneous severe congestion. These characteristics demand congestion control algorithms that simultaneously achieve high throughput and low latency.

Traditional loss-based algorithms (NewReno [2], CUBIC [3]) treat packet loss as a congestion signal, suffering from: (1) delayed congestion signals—buffers accumulate substantial data before overflow; (2) buffer bloat [4]—persistent high buffer occupancy; (3) inability to differentiate congestion severity; (4) slow convergence in high-BDP networks. Delay-based algorithms (Vegas [5], BBR [6]) detect congestion earlier via RTT monitoring, but face unstable measurements and poor fairness when coexisting with loss-based flows [7]. ECN [8] provides more precise congestion signals, and DCTCP [9] leverages ECN marking ratios for fine-grained adjustment, but existing algorithms still underutilize ECN signals.

Reinforcement learning (RL) has brought new perspectives to congestion control [12]. Aurora [13] uses deep RL to learn sending rate policies, and Orca [14] dynamically adjusts AIMD parameters via RL. However, existing RL-based schemes suffer from incomplete state space design (typically only 4–6 dimensions), lack hierarchical utilization of ECN signals, and exhibit high decision latency due to neural network inference.

This paper presents the Lark algorithm. The name “Lark” symbolizes the design goals of *High Throughput* and *Low Latency*, evoking a lark that is agile, responsive, and swift. The main contributions are:

- **15-dimensional observation space:** For the first time, ECN state, congestion control state machine state, and congestion event types are incorporated into the RL state representation, providing comprehensive protocol stack awareness.
- **Two-level priority congestion detection with consecutive decrease protection:** A hierarchical mechanism distinguishes loss, timeout, and ECN signals, with a consecutive decrease counter ($D_{\max} = 3$) that freezes the window after sustained multiplicative reductions.
- **Differentiated congestion response:** Type-specific window retention factors (loss: 0.70, ECN: 0.85, timeout: 0.50) precisely match congestion severity, with BDP-anchored slow-start threshold recovery ($ssthresh = \max(new_wnd, BDP)$).
- **RL-driven adaptive parameter tuning:** The mul-

multiplicative increase factor $\alpha \in [1.05, 1.50]$ is dynamically adjusted via three-level RTT gradient, reward exponential moving average (EMA), and sustained growth acceleration.

- **Hybrid rate-window fusion control:** Windowed max-filter BDP estimation combined with pipeline utilization-aware acceleration for both slow start and congestion avoidance phases.

Extensive experiments on the ns-3 platform demonstrate that Lark achieves throughput comparable to traditional optimal algorithms across 9 representative data center scenarios while reducing latency by 70% to 81%.

2 Related Work

2.1 Loss-based Algorithms

TCP NewReno [2] employs the AIMD strategy, defined in RFC 6582. CUBIC [3], the default in Linux since 2.6.19, uses a cubic function model $W(t) = C(t - K)^3 + W_{max}$ for window adjustment, where $K = \sqrt[3]{W_{max} \cdot \beta / C}$. CUBIC’s growth rate is RTT-independent, improving convergence in high-BDP networks. However, both algorithms treat packet loss as the sole congestion signal, suffering from delayed signals and buffer bloat in modern data centers.

2.2 Delay-based Algorithms

Vegas [5] compares expected and actual throughput to infer queuing delay: $\Delta = cwnd / BaseRTT - cwnd / RTT$. BBR [6] explicitly estimates bottleneck bandwidth ($BtlBw$) and minimum RTT (RTT_{prop}), targeting the BDP for rate adjustment: $pacing_rate = pacing_gain \times BtlBw$. Copa [16] introduces competitive mode detection. However, RTT measurements are susceptible to interference, and fairness degrades significantly when coexisting with loss-based algorithms—BBR flows may lose over 30% throughput competing with CUBIC [7].

2.3 ECN-based Algorithms

DCTCP [9] uses the ECN marking proportion for fine-grained window adjustment: $cwnd = cwnd \times (1 - \alpha/2)$, where α is the EWMA of the ECN marking ratio. HPCC [10] leverages precise congestion information from In-band Network Telemetry (INT). TIMELY [11] uses RTT gradients for rate control in data centers. However, existing algorithms utilize ECN along a single signal dimension only.

2.4 RL-based Methods

Aurora [13] trains a neural network policy with PPO, using a reward function $r = throughput - \delta \cdot delay - \lambda \cdot loss$. Orca [14] uses RL to dynamically adjust AIMD parameters. PCC [15] maximizes a utility function $U(x) = T \cdot x^\alpha - \delta \cdot L \cdot x - \beta \cdot D \cdot x$ via online learning. Remy [17] employs offline optimization to generate congestion control

rules. These schemes typically use only 4–6 dimensional state spaces, omitting ECN and CA state information.

Lark advances beyond existing work in: (1) a 15-dimensional observation space providing complete protocol stack awareness; (2) multi-signal fusion rather than single-signal detection; (3) differentiated rather than uniform congestion response; (4) rule-based real-time decisions (~ 0.1 ms) rather than GPU-dependent neural network inference (~ 1 – 10 ms).

3 Lark Algorithm Design

3.1 System Architecture

Lark comprises four modules: (1) a **network simulation module** built on ns-3 [18] with multi-threaded parallel architecture for simulation acceleration; (2) a **congestion control environment module** based on OpenGym [19] that collects the 15-dimensional state at TCP callback points (GetSsThresh, IncreaseWindow, PktsAacked, CongestionStateSet, CwndEvent); (3) an **intelligent decision module** implemented in Python executing multi-signal fusion detection, differentiated response, adaptive tuning, and fusion control; (4) an **inter-process communication module** using ZeroMQ with Protocol Buffers serialization.

3.2 RL Problem Formulation

Lark formulates congestion control as a Markov Decision Process $\langle S, A, P, R, \gamma \rangle$. The state space S is the 15-dimensional observation space. The action space A directly outputs new ssthresh and cwnd values, providing the finest control granularity. Lark adopts a **rule-based deterministic policy** rather than a neural network policy, trading some representational capacity for real-time performance (~ 0.1 ms decision latency) and interpretability.

3.3 15-Dimensional Observation Space

Lark designs a comprehensive observation space containing 15 parameters, as shown in Table 1.

The 15 parameters span six categories: *connection identification* (socketUuid, nodeId) for multi-flow state isolation; *temporal information* (envType, simTime) for delivery rate computation; *window state* (ssthresh, cwnd) as core congestion control variables; *transmission metrics* (segSize, segmentsAacked, bytesInFlight) enabling pipeline utilization calculation $u = bytesInFlight / cwnd$; *delay measurements* (lastRtt, minRtt) for BDP estimation and congestion assessment via the inflation ratio $r = lastRtt / minRtt$; and *congestion control state* (callbackType, caState, caEvent, ecnState) providing complete protocol stack awareness.

Compared to the 4–6 dimensional state spaces of existing schemes, Lark is the first to incorporate caState (CA_OPEN / DISORDER / CWR / RECOVERY / LOSS), caEvent (including ECN_IS_CE), and ecnState (including ECN_CE_RCVD) into the RL state repre-

Table 1: Lark 15-Dimensional Observation Space

Idx	Parameter	Description
0	socketUuid	Socket unique identifier
1	envType	Environment type
2	simTime	Simulation time (μ s)
3	nodeId	Node identifier
4	ssthresh	Slow start threshold (B)
5	cwnd	Congestion window (B)
6	segSize	Segment size (B)
7	segmentsAked	Segments acknowledged
8	bytesInFlight	Bytes in flight
9	lastRtt	Latest RTT (μ s)
10	minRtt	Minimum RTT (μ s)
11	callbackType	Callback type
12	caState	CA state
13	caEvent	CA event
14	ecnState	ECN state

sensation. These three parameters provide complete congestion state information from the protocol stack, enabling the agent to distinguish between different types and severity levels of congestion.

3.4 Multi-Signal Fusion Congestion Detection

Lark classifies congestion signals into three types—*packet loss*, *timeout*, and *ECN*—organized in a two-level priority hierarchy. Level-1 signals (highest priority) are triggered when *callbackType* = 0 (the GETSSSTHRESH callback), indicating that the protocol stack has detected a congestion event. Within Level-1, timeout is distinguished from ordinary packet loss by inspecting *caState*: if *caState* = CA_LOSS, the event is classified as a retransmission timeout (RTO), typically indicating consecutive packet losses or path failure; otherwise, the event is an ordinary loss triggered by fast retransmit. Level-2 signals correspond to ECN congestion experienced, detected via direct protocol stack state inspection: *ecnState* $\in$ {ECN_CE_RCVD, ECN_ECE_RCVD}.

A critical safeguard is the **consecutive decrease protection** mechanism. The agent maintains a per-flow counter d that increments on every congestion response and resets to zero on every window increase. When d exceeds $D_{\max} = 3$, the congestion response is suppressed—the current window is frozen rather than multiplicatively reduced. This prevents the “death spiral” in which successive multiplicative decreases shrink the window to impractically small values (e.g., $0.70^3 \approx 34.3\%$ after three loss responses). The detection procedure is formalized in Algorithm 1.

Algorithm 1 Multi-Signal Fusion Congestion Detection

Require: Observation *obs*, per-flow state *state*
Ensure: (*is_congested*, *type*)

```

1: // Level-1: Loss and timeout (highest priority)
2: if obs.callbackType = 0 then
3:   state.loss_count += 1
4:   if obs.caState = CA_LOSS then
5:     return (TRUE, TIMEOUT)
6:   else
7:     return (TRUE, LOSS)
8:   end if
9: end if
10: // Level-2: ECN state inspection
11: if obs.ecnState  $\in$  {CE_RCVD, ECE_RCVD} then
12:   state.ecn_count += 1
13:   return (TRUE, ECN)
14: end if
15: return (FALSE, NONE)

```

3.5 Differentiated Congestion Response

Traditional algorithms uniformly apply $\rho = 0.5$ (window halving) to all congestion signals, failing to distinguish congestion severity. Lark applies type-specific window retention factors ρ :

$$new_cwnd = \max(\rho \cdot cwnd, 4 \times MSS), \quad \rho = \begin{cases} 0.70 & \text{LOSS} \\ 0.85 & \text{ECN} \\ 0.50 & \text{TIMEOUT} \end{cases} \quad (1)$$

The design rationale is grounded in signal temporal characteristics and severity. ECN is an early signal, triggered when queue occupancy exceeds a low marking threshold ($\sim 30\%$); a moderate 15% window reduction suffices to alleviate incipient congestion while preserving throughput. Packet loss indicates more severe congestion—the queue has overflowed—warranting a 30% reduction. Timeout (*caState* = CA_LOSS) represents the most severe condition, typically caused by consecutive packet losses or path failure, and thus receives the most aggressive 50% reduction, matching the classical halving response.

The slow-start threshold is updated with a BDP-anchored lower bound:

$$ssthresh = \max(new_cwnd, BDP) \quad (2)$$

This ensures that the threshold never drops below the estimated bandwidth-delay product, enabling faster post-congestion recovery by avoiding unnecessarily conservative slow-start re-entry.

The complete differentiated response, including consecutive decrease protection, is presented in Algorithm 2.

Algorithm 2 Differentiated Congestion Response

Require: Current *cwnd*, congestion type *type*, per-flow state *state*, BDP estimate *bdp*

Ensure: (*new_cwnd*, *new_ssthresh*)

```

1: min_cwnd  $\leftarrow \max(4 \times MSS, 1)$ 
2: state.consecutive_decreases += 1
3: state.consecutive_increases  $\leftarrow 0$ 
4: // Consecutive decrease protection
5: if state.consecutive_decreases >  $D_{\max}$  then
6:   return ( $\max(cwnd, min\_cwnd)$ ,  $\max(cwnd, min\_cwnd)$ )
7: end if
8: // Type-specific retention factor
9: if type = TIMEOUT then
10:   $\rho \leftarrow 0.50$ 
11: else if type = ECN then
12:   $\rho \leftarrow 0.85$ 
13: else
14:   $\rho \leftarrow 0.70$  {LOSS (default)}
15: end if
16: new_cwnd  $\leftarrow \max(\lfloor \rho \cdot cwnd \rfloor, min\_cwnd)$ 
17: new_ssthresh  $\leftarrow \max(new\_cwnd, \lfloor bdp \rfloor, min\_cwnd)$ 
18: return (new_cwnd, new_ssthresh)

```

3.6 Adaptive Parameter Tuning

Lark dynamically adjusts the multiplicative increase factor $\alpha \in [1.05, 1.50]$ (default initial value $\alpha_0 = 1.20$) based on three complementary feedback signals, enhancing robustness beyond any single indicator.

Three-level RTT gradient. The RTT inflation ratio $r = lastRtt/minRtt$ serves as a proxy for queuing delay. We partition r into three regimes providing fine-grained control: (i) $r < 1.3$: negligible queuing, $\alpha += 0.03$ and the consecutive increase counter g is incremented; (ii) $1.3 \leq r < 2.0$: moderate queuing, $\alpha += 0.01$; (iii) $r > 3.0$: severe queuing, $\alpha -= 0.02$ and g is reset to zero.

Reward EMA trend. An exponential moving average of the reward signal captures long-term performance trends: $\bar{r}_t = (1 - \eta)\bar{r}_{t-1} + \eta \cdot r_t$ with smoothing factor $\eta = 0.1$. When $\bar{r} > 2.0$ (sustained positive performance), $\alpha += 0.01$; when $\bar{r} < -5.0$ (sustained degradation), $\alpha -= 0.01$.

Sustained growth acceleration. When the consecutive increase counter exceeds 5 ($g > 5$), indicating a prolonged period of stable growth, $\alpha += 0.02$ and g is reset, implementing a “progressive acceleration” policy.

All adjustments employ small step sizes (0.01–0.03) to avoid instability. The lower bound 1.05 guarantees a minimum growth rate, while the upper bound 1.50 prevents overly aggressive probing. From a control-theoretic perspective, α adjustment is analogous to dynamically tuning the controller gain K in response to the observed delay τ (RTT), satisfying the stability condition $K\tau < \pi/2$. The complete procedure is presented in Algorithm 3.

Algorithm 3 RL-Driven Adaptive Parameter Tuning

Require: Current α , observation *obs*, consecutive growth count g , reward EMA $\bar{r}$, current reward r_t

Ensure: Updated α , g , $\bar{r}$

```

1:  $\bar{r} \leftarrow (1 - \eta) \cdot \bar{r} + \eta \cdot r_t$  {Update reward EMA,  $\eta = 0.1$ }
2: ratio  $\leftarrow obs.lastRtt / obs.minRtt$ 
3: // Three-level RTT gradient
4: if ratio < 1.3 then
5:   $\alpha \leftarrow \alpha + 0.03$ ;  $g \leftarrow g + 1$ 
6: else if ratio < 2.0 then
7:   $\alpha \leftarrow \alpha + 0.01$ 
8: else if ratio > 3.0 then
9:   $\alpha \leftarrow \alpha - 0.02$ ;  $g \leftarrow 0$ 
10: end if
11: // Reward EMA trend
12: if  $\bar{r} > 2.0$  then
13:   $\alpha \leftarrow \alpha + 0.01$ 
14: else if  $\bar{r} < -5.0$  then
15:   $\alpha \leftarrow \alpha - 0.01$ 
16: end if
17: // Sustained growth acceleration
18: if  $g > 5$  then
19:   $\alpha \leftarrow \alpha + 0.02$ ;  $g \leftarrow 0$ 
20: end if
21:  $\alpha \leftarrow \text{clamp}(\alpha, 1.05, 1.50)$ 

```

3.7 Fusion Control Strategy

Lark combines BDP-based rate control with current window-based conservative control. The BDP is estimated via a **windowed max-filter**: the delivery rate $DR = segmentsAked \times segSize / lastRtt$ is appended to a sliding window of capacity $W_{bw} = 20$, and the maximum over the window yields the bottleneck bandwidth estimate $BW_{\max}$. The minimum RTT is tracked globally via a sliding window of capacity $W_{rtt} = 40$. The BDP is then $BDP = BW_{\max} \times RTT_{\min}$. When the BDP estimate is unavailable (e.g., during connection startup), BDP falls back to $\max(cwnd, 1)$. The estimation procedure is formalized in Algorithm 4.

Algorithm 4 Windowed Max-Filter BDP Estimation

Require: Per-flow state *state*, observation *obs*

Ensure: BDP estimate *bdp*

```

1: dr  $\leftarrow obs.segAked \times obs.segSize / obs.lastRtt$ 
2: state.bw_samples.append(dr) {Sliding window, capacity  $W_{bw} = 20$ }
3:  $BW_{\max} \leftarrow \max(state.bw\_samples)$ 
4:  $RTT_{\min} \leftarrow \min(state.min\_rtt, obs.minRtt)$ 
5: state.min_rtt  $\leftarrow RTT_{\min}$ 
6: bdp  $\leftarrow BW_{\max} \times RTT_{\min}$ 
7: if bdp  $\leq 0$  then
8:  bdp  $\leftarrow \max(obs.cwnd, 1)$  {Fallback to current window}
9: end if
10: return bdp

```

Slow start phase ($cwnd < ssthresh$ and

$in_slow_start = \text{TRUE}$): The target window is $target = \max(2 \times BDP, 10 \times MSS)$. The base increment is $\Delta = segmentsAked \times MSS$ (standard exponential growth). When the window is far below the BDP estimate ($cwnd < 0.3 \times BDP$), the increment doubles to $2 \times segmentsAked \times MSS$ for rapid bandwidth exploration. The window is capped at $target$; upon reaching $target$, slow start exits and $in_slow_start \leftarrow \text{FALSE}$.

Congestion avoidance phase ($cwnd \geq ssthresh$): The new window is set directly to the fusion target:

$$new_cwnd = \max(\alpha \times BDP, cwnd) + \gamma \times MSS \quad (3)$$

where $\gamma = 2.0$ provides a constant additive growth bias of two segments per ACK event. The BDP-guided target ensures that the window tracks the estimated path capacity, while the max operator prevents unnecessary regression below the current operating point.

Pipeline utilization-aware acceleration. The pipeline utilization $u = bytesInFlight / cwnd$ quantifies how fully the network pipe is utilized. When $u < 0.7$, an additional $1 \times MSS$ is added; when $u < 0.4$, a further $2 \times MSS$ is added (cumulative $3 \times MSS$ total), ensuring rapid adaptation when the pipe is underutilized.

Window bounds are $cwnd \in [4 \times MSS, \max(10 \times BDP, 200 \times MSS)]$. On each non-congestion event, the consecutive decrease counter is reset to zero. The complete fusion control is presented in Algorithm 5.

3.8 Per-Flow Independent State Management

Lark uses socketUuid as the key with $O(1)$ hash table lookup to maintain independent state for each TCP connection. Each per-flow state comprises four sub-modules: (1) a *bandwidth estimation module*—a delivery rate sliding window (capacity $W_{bw} = 20$) and an RTT sample queue (capacity $W_{rtt} = 40$) for windowed max-filter bandwidth estimation and global minimum RTT tracking; (2) *congestion control counters*—consecutive decrease counter d (for protection), consecutive increase counter g (for acceleration trigger), and cumulative loss/ECN event counts; (3) *performance metrics*—throughput EMA, current BDP estimate, previous window and timestamp; (4) *adaptive parameters*— α (initial 1.20), $\gamma = 2.0$, and a slow-start flag.

State management follows lazy initialization (created on first access) and incremental update (only changed fields are modified). Each flow has its own independent TcpLark agent instance managed via a per-socketUuid dictionary. A globally shared reward EMA (smoothing factor $\eta = 0.1$) captures cross-flow aggregate performance trends. Per-flow independent management ensures that in multi-flow concurrent scenarios, each flow receives differentiated control without cross-flow interference.

Algorithm 5 Hybrid Rate-Window Fusion Control

Require: Observation obs , parameters α, γ , BDP estimate bdp , per-flow state $state$

Ensure: ($new_cwnd, new_ssthresh$)

```

1:  $state.consecutive\_decreases \leftarrow 0$  {Reset on growth}
2:  $state.consecutive\_increases += 1$ 
3:  $u \leftarrow obs.bytesInFlight / obs.cwnd$ 
4: if  $obs.cwnd < obs.ssthresh$  and  $state.in\_slow\_start$ 
   then
5:    $target \leftarrow \max(2 \times bdp, 10 \times MSS)$ 
6:    $\Delta \leftarrow obs.segAked \times MSS$ 
7:   if  $bdp > 0$  and  $obs.cwnd < 0.3 \times bdp$  then
8:      $\Delta \leftarrow 2 \times obs.segAked \times MSS$ 
9:   end if
10:   $new\_cwnd \leftarrow \min(obs.cwnd + \Delta, target)$ 
11:  if  $new\_cwnd \geq target$  then
12:     $state.in\_slow\_start \leftarrow \text{FALSE}$ 
13:  end if
14: else
15:   // Congestion avoidance
16:    $new\_cwnd \leftarrow \max(\alpha \times bdp, obs.cwnd) + \gamma \times MSS$ 
17: end if
18: // Pipeline utilization-aware acceleration
19: if  $u < 0.7$  then
20:    $new\_cwnd += 1 \times MSS$ 
21: end if
22: if  $u < 0.4$  then
23:    $new\_cwnd += 2 \times MSS$ 
24: end if
25:  $new\_cwnd \leftarrow \text{clamp}(new\_cwnd, 4 \times MSS, \max(10 \times bdp, 200 \times MSS))$ 
26:  $new\_ssthresh \leftarrow obs.ssthresh$ 
27: return ( $new\_cwnd, new\_ssthresh$ )

```

4 Experimental Evaluation

4.1 Experimental Setup

Platform: ns-3.40 + OpenGym [19], Intel i7-10700K 8-core @ 3.8 GHz, 32 GB DDR4 RAM, 512 GB NVMe SSD, Ubuntu 20.04 LTS. Python 3.8.10, NumPy 1.21, ZeroMQ 4.3.4.

Topology: Classic dumbbell topology with 3 sender/receiver pairs sharing a bottleneck link. ECN enabled (RED queue, marking threshold 30%), SACK enabled, MSS 1460 bytes, initial window 10 MSS (RFC 6928), simulation duration 20 seconds, random seed 42 for reproducibility.

Baselines: NewReno [2], CUBIC [3], BBR [6].

Metrics: Throughput (Mbps), average latency (ms), Jain fairness index [20], bandwidth utilization (%), loss rate.

4.2 Scenario Design

We design 9 representative scenarios spanning intra-rack, cross-Pod, and WAN environments, as shown in Table 2.

Table 2: 9 Experimental Scenario Configurations

#	Type	BN BW	BN Delay
1	Intra-rack high-speed	10 Gbps	2 μ s
2	4:1 oversub. 10G	2.5 Gbps	5 μ s
3	4:1 oversub. 40G	10 Gbps	5 μ s
4	Moderate cong. (5:1)	2 Gbps	5 μ s
5	Heavy cong. (10:1)	1 Gbps	5 μ s
6	Cross-Pod 10G	10 Gbps	50 μ s
7	Cross-Pod 20G	20 Gbps	50 μ s
8	Cross-DC WAN	1 Gbps	5 ms
9	Symmetric low BW	1 Gbps	20 μ s

4.3 Main Results

Table 3 summarizes the experimental results across all scenarios.

Throughput analysis. Across all 9 scenarios, Lark’s throughput is comparable to NewReno/CUBIC, with gaps not exceeding 3% and bandwidth utilization exceeding 99%. In the heavy congestion scenario (Scenario 5, 10:1 oversubscription), Lark achieves the identical 1021 Mbps throughput as NewReno/CUBIC, demonstrating full bottleneck utilization even under severe congestion. BBR performs poorly in data center scenarios, with throughput typically around 50% of bottleneck bandwidth, dropping to 3–6% in cross-Pod scenarios due to bandwidth probing mechanism failures under multi-flow competition.

Latency analysis. In data center internal scenarios (Scenarios 1–7, 9), Lark reduces latency from the 1.9–2.8 ms range of traditional algorithms to 0.44–0.84 ms, a reduction of 70.1% to 81.1%. Scenario 3 (4:1 oversubscription 40G) achieves the most significant reduction at

81.1%. In the WAN scenario, propagation delay dominates, yielding a 13.6% latency reduction, as propagation delay is incompressible.

BBR’s limitations. Although BBR achieves very low latency, it does so at severe throughput cost. In multi-flow competitive data center environments, BBR’s bandwidth probing mechanism fails to converge adequately—throughput drops below 6% of bottleneck bandwidth in cross-Pod scenarios, rendering it impractical for deployment.

4.4 In-Depth Analysis

4.4.1 Convergence

Table 4 compares convergence behavior in Scenario 2.

Lark converges fastest (0.8 s to 90% stable throughput) with the smallest steady-state fluctuation ($\pm 3\%$), owing to: (1) BDP-guided target window via windowed max-filter estimation; (2) $2 \times BDP$ slow-start target with accelerated exploration below $0.3 \times BDP$; (3) three-level RTT gradient-driven α adaptation that aggressively increases α when queuing delay is negligible. NewReno converges slowest due to its linear AIMD growth. BBR exhibits the largest fluctuation ($\pm 15\%$) due to periodic bandwidth probing.

4.4.2 Queue Behavior

In Scenario 2, NewReno/CUBIC maintain an average queue of ~ 120 packets (fluctuating in [50, 200]), while Lark maintains only ~ 8 packets (fluctuating in [2, 15]). Lower queue occupancy directly translates to reduced queuing delay. For a 2.5 Gbps bottleneck with 1500-byte packets:

$$Q_{\text{delay}} = \frac{Q_{\text{len}} \times 1500 \times 8}{2.5 \times 10^9} \quad (4)$$

NewReno/CUBIC: $Q_{\text{delay}} \approx 0.576$ ms; Lark: $Q_{\text{delay}} \approx 0.038$ ms, consistent with measured latency data.

4.4.3 ECN Utilization Efficiency

Table 5 compares ECN-related statistics in Scenario 2.

Lark’s ECN-marked packet count (1,245) is significantly lower than NewReno (3,567) and CUBIC (3,892), because Lark maintains lower queue occupancy, reducing ECN marking opportunities. Lark suffers only 12 packet losses (vs. NewReno: 156), as timely ECN response prevents queue overflow. Each congestion response reduces the window by only 15% (vs. 50% for NewReno), demonstrating the effectiveness of differentiated response with the consecutive decrease protection mechanism preventing over-reduction.

4.4.4 Fairness

We test fairness with 10 concurrent flows on a 2.5 Gbps bottleneck, as shown in Table 6.

Lark achieves a Jain fairness index of 0.98, close to NewReno (0.99) and better than CUBIC (0.97). BBR exhibits poor fairness (0.89) with a max-to-min ratio of 2.27. Lark’s per-flow independent state management

Table 3: Throughput and Latency Comparison Across 9 Scenarios

Scenario	Lark		NewReno		CUBIC		BBR		Lat. Red. vs NR
	Tput	Lat	Tput	Lat	Tput	Lat	Tput	Lat	
1: Intra-rack 10G	10193	0.46	10213	2.30	10213	2.29	2.3	0.01	80.1%
2: 4:1 OS 10G	2552	0.53	2553	2.33	2553	2.45	1299	0.02	77.3%
3: 4:1 OS 40G	9931	0.44	10213	2.30	10213	2.30	7107	0.01	81.1%
4: Moderate cong.	2042	0.61	2042	2.40	2042	2.49	1037	0.04	74.5%
5: Heavy cong.	1021	0.84	1021	2.80	1021	2.76	522	0.07	70.1%
6: Cross-Pod 10G	10109	0.49	10213	2.31	10211	2.40	600	0.06	78.9%
7: Cross-Pod 20G	19856	0.51	20408	1.90	20404	2.21	604	0.06	73.0%
8: WAN	1012	5.82	1016	6.74	1017	7.29	1019	9.08	13.6%
9: Sym. low BW	1018	0.74	1021	2.83	1021	2.78	517	0.07	73.7%

Tput: Throughput (Mbps), Lat: Latency (ms), NR: NewReno, OS: Oversubscription, BN: Bottleneck

Table 4: Convergence Time Comparison (Scenario 2)

Algo.	90% Conv.	99% Conv.	Fluctuation
Lark	0.8 s	1.2 s	$\pm 3\%$
NewReno	2.5 s	4.0 s	$\pm 8\%$
CUBIC	1.5 s	2.5 s	$\pm 6\%$
BBR	1.0 s	1.8 s	$\pm 15\%$

Table 5: ECN and Congestion Response Statistics (Scenario 2)

Metric	Lark	NR	CU	BBR
ECN-marked pkts	1,245	3,567	3,892	823
Packet losses	12	156	178	45
Cong. responses	89	156	178	67
Wnd. reduction	15%	50%	47%	25%

Table 6: Fairness Test Results (10 Concurrent Flows)

Algo.	Jain	Max (Mbps)	Min (Mbps)	Ratio
Lark	0.98	268.5	241.2	1.11
NewReno	0.99	261.3	248.7	1.05
CUBIC	0.97	275.2	232.1	1.19
BBR	0.89	198.5	87.3	2.27

ensures that differentiated control does not compromise fairness.

4.4.5 Mixed Traffic

In a mixed scenario with 3 long flows (20 s bulk transfers) and 5 short flows per second (0.1 s each) on a 2 Gbps bottleneck:

Table 7: Mixed Traffic Test Results

Metric	Lark	NR	CU	BBR
Long flow Tput (Mbps)	612.5	625.8	628.3	412.3
Short FCT (ms)	12.5	45.3	42.1	15.8
Short P99 FCT (ms)	28.3	125.6	118.2	35.2
Link util. (%)	98.2	99.1	99.3	72.5

Lark’s short flow completion time (12.5 ms) is $3.6\times$ better than NewReno (45.3 ms). The P99 tail latency improvement is even more dramatic: 28.3 ms vs. 125.6 ms ($4.4\times$). Long flow throughput remains comparable (612.5 vs. 625.8 Mbps, only 2.1% gap). Low queue occupancy is the primary driver—reduced queuing delay enables short flows to complete faster, which is critically important for latency-sensitive workloads such as distributed databases and real-time analytics.

5 Discussion

5.1 Design Rationale

The ECN retention factor of 0.85 is chosen because ECN is an early congestion signal—the queue has only exceeded a low marking threshold ($\sim 30\%$), and a moderate 15% reduction suffices to alleviate incipient congestion; aggressive reduction would cause unnecessary throughput loss. The packet loss retention factor of 0.70 is higher than the traditional 0.50 because ECN’s early warning mechanism has already initiated congestion mitigation in ECN-capable networks, reducing the need for aggressive loss response. The timeout retention factor of 0.50 matches the traditional halving response, reflecting that RTO typically indicates severe conditions such as consecutive losses or path failures. The consecutive

decrease protection ($D_{\max} = 3$) prevents the “death spiral” phenomenon: after three consecutive loss responses, the window is only $0.70^3 \approx 34.3\%$ of its original value, already below the classical single halving; further reduction yields diminishing marginal benefit.

The adaptive parameter tuning employs three complementary feedback signals to enhance robustness, as any single indicator may be subject to transient noise. The three-level RTT gradient (thresholds at 1.3, 2.0, and 3.0) provides fine-grained control across the full spectrum of queuing delay. The reward EMA ($\eta = 0.1$) captures long-term performance trends that instantaneous RTT measurements may miss. The sustained growth trigger ($g > 5$) implements a “progressive acceleration” policy that rewards stability. Gradual adjustment with small step sizes (0.01–0.03) avoids instability, and boundary constraints $\alpha \in [1.05, 1.50]$ prevent anomalous behavior.

5.2 Comparison with Existing Approaches

Table 8 presents a detailed feature comparison.

Table 8: Detailed Feature Comparison

Feature	Lark	Aurora	DCTCP	BBR
State dims	15	10	–	–
ECN util.	Yes	No	Yes	No
CA state util.	Yes	No	No	No
Decision mech.	Rule	DNN	Formula	Model
Interpretable	Yes	No	Yes	Partial
Training req.	None	Heavy	None	None
Decision lat.	0.1ms	1–10ms	<0.1ms	<0.1ms
Param. adapt.	Yes	Fixed	No	No
Multi-signal	Yes	No	No	No
Diff. response	Yes	No	No	No

vs. Aurora [13]. Aurora uses a 10-dimensional state space with deep neural network inference incurring 1–10 ms decision latency. Lark’s 15-dimensional state space provides richer protocol stack information. Lark’s rule-based decisions require ~ 0.1 ms, require no training, and are interpretable, facilitating debugging and deployment.

vs. DCTCP [9]. DCTCP uses ECN marking proportions for continuous adjustment ($cwnd = cwnd \times (1 - \alpha/2)$), applying a uniform formula to all ECN signals. Lark directly inspects the protocol stack ECN state, applies type-specific retention factors (loss 0.70 / ECN 0.85 / timeout 0.50), and introduces consecutive decrease protection ($D_{\max} = 3$). Additionally, Lark leverages caState and caEvent information with adaptive α tuning, enabling richer signal utilization than DCTCP’s single-dimension approach.

vs. BBR [6]. BBR targets BDP for rate-based control via periodic bandwidth probing, which causes throughput oscillations and fails under multi-flow competition in data centers (3–6% utilization in cross-Pod scenarios). Lark achieves >99% bandwidth utilization while

maintaining significantly lower latency than loss-based algorithms, without periodic probing.

5.3 Complexity Analysis

Time complexity: Per-decision $O(1)$, comprising observation processing $O(15)$, congestion detection $O(1)$, parameter tuning $O(1)$, and window computation $O(1)$. On Intel i7-10700K, mean decision latency is 0.08 ms with P99 of 0.15 ms.

Space complexity: Per-flow $O(W_{bw} + W_{rtt})$ where $W_{bw} = 20$ and $W_{rtt} = 40$ are the sliding window capacities, plus constant-size counters and parameters, totaling approximately 1.2 KB per flow. For 10,000 concurrent flows, total memory is ~ 24 MB including Python runtime and ZeroMQ overhead.

CPU utilization: At typical data center rates ($\sim 1,000$ decisions/s for 10 Gbps), single-core utilization is $\sim 1.2\%$, significantly lower than deep learning approaches.

5.4 Sensitivity Analysis

We evaluate sensitivity to key parameters in Scenario 2. The loss retention factor was tested from 0.50 to 0.90: lower values (0.50–0.60) reduce throughput by 2–3% but slightly improve latency; higher values (0.80–0.90) marginally increase throughput but significantly increase latency and packet losses. The ECN retention factor was tested from 0.70 to 0.95: the default 0.85 achieves a favorable balance between throughput preservation and congestion responsiveness. The consecutive decrease protection threshold $D_{\max}$ was tested from 1 to 10: $D_{\max} = 1$ is overly conservative (latency increases by $\sim 40\%$); $D_{\max} = 10$ offers negligible additional protection over no limit; the default $D_{\max} = 3$ provides robust performance. The three-level RTT gradient (1.3/2.0/3.0) was compared against a simpler two-level scheme (1.5/3.0): the three-level design reduces latency by $\sim 8\%$ in moderate congestion scenarios by providing finer-grained α control in the intermediate queuing regime. The default parameter configuration (loss: 0.70, ECN: 0.85, timeout: 0.50, $D_{\max} = 3$, $\alpha \in [1.05, 1.50]$) achieves robust performance across all tested scenarios.

5.5 Limitations and Future Directions

Limitations. (1) Latency optimization in WAN scenarios is limited (13.6%) because propagation delay is incompressible; queuing delay isolation techniques may help. (2) Parameters such as the consecutive decrease protection threshold $D_{\max}$, RTT gradient boundaries (1.3/2.0/3.0), and retention factors may require environment-specific tuning; meta-learning [17] or Bayesian optimization could automate this process. (3) Lark’s full effectiveness depends on ECN support in the network infrastructure.

Future directions. (1) Deep RL or meta-learning for automatic parameter tuning, potentially discovering superior control strategies through neural network poli-

cies, with model distillation for deployment. (2) Linux kernel implementation (porting Python to C) and real data center deployment via A/B testing. (3) QUIC protocol integration, leveraging user-space implementation for flexible deployment and multi-stream optimization. (4) Multi-objective optimization extensions including explicit fairness optimization, energy efficiency for edge computing, and customizable objective weights. (5) Cross-layer optimization coordinating with application, link, and scheduling layers.

6 Conclusion

This paper presents Lark—a reinforcement learning-based multi-signal fusion adaptive congestion control algorithm. Through a comprehensive 15-dimensional observation space incorporating ECN state, CA state, and congestion events, two-level priority congestion detection with consecutive decrease protection ($D_{\max} = 3$), differentiated window retention factors (loss: 0.70, ECN: 0.85, timeout: 0.50) with BDP-anchored threshold recovery, RL-driven adaptive parameter tuning via three-level RTT gradient and reward EMA ($\alpha \in [1.05, 1.50]$), and a hybrid rate-window fusion control strategy with windowed max-filter BDP estimation and pipeline utilization-aware acceleration, Lark achieves throughput comparable to traditional optimal algorithms (gap <1%) while reducing latency by 70% to 81% (from 1.9–2.8 ms to 0.44–0.84 ms) across 9 representative data center network scenarios. In-depth analysis confirms Lark’s advantages in convergence speed (0.8 s to 90% throughput), queue management (~ 8 vs. ~ 120 packets), ECN efficiency (12 vs. 156 packet losses), fairness (Jain index 0.98), and mixed traffic performance ($3.6\times$ improvement in short flow completion time). Lark successfully realizes its “High Throughput, Low Latency” design objectives with rule-based $O(1)$ decisions at ~ 0.1 ms latency, providing an efficient, interpretable, and adaptive solution for data center network congestion control.

References

- [1] Benson T, Akella A, Maltz D A. Network traffic characteristics of data centers in the wild. In Proc. ACM IMC, 2010: 267–280.
- [2] Floyd S, Henderson T, Gurtov A. The NewReno modification to TCP’s fast recovery algorithm. RFC 6582, 2012.
- [3] Ha S, Rhee I, Xu L. CUBIC: a new TCP-friendly high-speed TCP variant. ACM SIGOPS OSR, 2008, 42(5): 64–74.
- [4] Gettys J, Nichols K. Bufferbloat: Dark buffers in the Internet. Commun. ACM, 2012, 55(1): 57–65.
- [5] Brakmo L S, Peterson L L. TCP Vegas: End to end congestion avoidance on a global Internet. IEEE JSAC, 1995, 13(8): 1465–1480.
- [6] Cardwell N, Cheng Y, Gunn C S, et al. BBR: Congestion-based congestion control. ACM Queue, 2016, 14(5): 20–53.
- [7] Hock M, Bless R, Zitterbart M. Experimental evaluation of BBR congestion control. IEEE ICNP, 2017: 1–10.
- [8] Ramakrishnan K, Floyd S, Black D. The addition of explicit congestion notification (ECN) to IP. RFC 3168, 2001.
- [9] Alizadeh M, Greenberg A, Maltz D A, et al. Data center TCP (DCTCP). ACM SIGCOMM, 2010, 40(4): 63–74.
- [10] Li Y, Miao R, Liu H H, et al. HPCC: High precision congestion control. ACM SIGCOMM, 2019: 44–58.
- [11] Mittal R, Lam V T, Dukkipati N, et al. TIMELY: RTT-based congestion control for the datacenter. ACM SIGCOMM, 2015, 45(4): 537–550.
- [12] Jay N, Rotman N, Godfrey B, et al. A deep reinforcement learning perspective on Internet congestion control. ICML, 2019: 3050–3059.
- [13] Jay N, Rotman N, Godfrey B, et al. Internet congestion control via deep reinforcement learning. NeurIPS, 2019.
- [14] Abbasloo S, Yen C Y, Chao H J. Classic meets modern: A pragmatic learning-based congestion control for the Internet. ACM SIGCOMM, 2020: 632–647.
- [15] Dong M, Li Q, Zarchy D, et al. PCC: Re-architecting congestion control for consistent high performance. USENIX NSDI, 2015: 395–408.
- [16] Arun V, Balakrishnan H. Copa: Practical delay-based congestion control for the Internet. USENIX NSDI, 2018: 329–342.
- [17] Winstein K, Balakrishnan H. TCP ex machina: Computer-generated congestion control. ACM SIGCOMM, 2013, 43(4): 123–134.
- [18] Riley G F, Henderson T R. The ns-3 network simulator. Modeling and Tools for Network Simulation. Springer, 2010: 15–34.
- [19] Gawłowicz P, Zubow A. ns-3 meets OpenAI Gym: The playground for machine learning in networking research. ACM MSWiM, 2019: 113–120.
- [20] Jain R K, Chiu D M W, Hawe W R. A quantitative measure of fairness and discrimination. DEC Research Report TR-301, 1984.